CITI BIKE REBALANCING EXPLORER · DOCUMENTATION SET V2

Document 04 — Lessons Learned: AI Collaboration & System-Design Process

How it was actually built

Reusable practices for AI-assisted development, for designing a small and honest data system, and for the product decisions that kept it credible — each drawn from something that actually happened while building this project.

Read last. This is the meta-layer over Documents 01–03 — the process, not the product.

Nature: concrete, transferable lessons with real examples, not generic best-practice boilerplate.

Prepared: July 2026 · independent portfolio project.

Part A — Working with an AI assistant

A1. Write standing rules the assistant cannot silently override

The single highest-leverage practice was a short, fixed set of project rules that override the model's defaults on every turn. They are not suggestions in a first prompt that decay over a long session — they are persistent, and the assistant is expected to stop and flag rather than pattern-match past them.

> **RULES THAT EARNED THEIR PLACE**
>
> **Never invent numbers** (label anything illustrative). · **Baseline-first** (a complex model is adopted only where it beats a plain one out-of-sample). · **No map-level equity filter** (a deliberate scope boundary, even though the reference implementation has one). · **UI default state must be set in JS, not implied by markup** (a bug class that recurred three times before it became a rule).

Why it works: a standing rule turns a judgment the human would otherwise re-make every session into a constraint the assistant enforces for itself — including catching itself pattern-matching a reference implementation toward the very thing the rule forbids.

A2. Explain the approach before writing the code

For any non-trivial change, the assistant explains the intended approach in a few sentences and waits for a go-ahead before implementing. This "learning-mode" gate caught mismatched assumptions cheaply — in conversation, before they became code that had to be reviewed and unwound — and kept the human in control of direction rather than reviewing finished work after the fact.

A3. One session = one unit of work, ending in tests and a written log

Each working session took on one module or one coherent change, ran the test suite at the end, and wrote what was built and what was learned into a durable log (`PROGRESS.md`). That log — not the chat history — is the real memory across sessions: it is where "we tried the arrivals/departures split and it lost" or "the depot data was researched and rejected" lives, so a later session does not re-litigate a settled question or silently re-introduce a rejected idea.

A4. Verify before claiming — at every layer, including your own comments

The most important discipline: nothing is reported as done or true until it is checked against reality. This showed up concretely again and again.

- **Behavioral tests, not code-reading.** UI changes were verified by driving a real headless browser and asserting the actual resulting DOM state, then confirming with screenshots — not by re-reading the diff and declaring it correct.

- **Screenshots catch what computed styles miss.** One chevron had a perfectly correct computed `content` / `color` / `width` — and was invisible, clipped by a parent's overflow from a missing `box-sizing`. Only a rendered screenshot showed it.
- **Check specific numbers, not "looks fine."** Two individually-reasonable layout changes (center the time bar; move the legend to its row) combined into a real 56–96 px overlap — found by measuring element rectangles at several window widths, which "it looked fine in one screenshot" would have missed.
- **Even self-authored claims get verified.** A code comment that asserted a crash was "confirmed by testing" was corrected before the test had actually run — the claim was walked back to honest wording, then the test was really run.

A5. Re-check inherited reasoning; do not treat an old caveat as permanent

Reasons rot. Several times, a documented exclusion turned out to rest on a mechanism that no longer existed:

- A weather preset (`hot_day`) had been excluded for years because of a specific artifact in a gradient-boosting partial-dependence curve — but that model had since been replaced by a direct regression and then deleted entirely. The stated reason could no longer occur; the preset was unblocked after confirming that against the current code and the real 435-value temperature range, not assumed still valid.
- A "no newer school dataset exists" caveat was re-verified against the live data portal (not re-read from an old note) and found still true — for five specific, checkable reasons.

Lesson: a years-old caveat deserves the same "check against current reality" treatment as any other claim. The alternative is a codebase that accumulates frozen justifications no one revalidates.

A6. Use a reference implementation as a diff target, never a copy source

A finished prior version existed, but the rule was to build independently and consult the reference only to produce a diff — "here is what mine does differently and which is better." That preserved the learning value of the rebuild and, more than once, surfaced that the reference's choice (e.g. its map-level equity filter, its module structure) was *not* the one to reproduce.

A7. Let the human's current intent override past rationale — including the assistant's own

When the human asked to narrow and center a control that an earlier session had deliberately made full-width, the right move was to make the change and note the tension — not to defend the earlier decision. A written rationale explains why a past choice was made; it is not a constraint on the person who owns the project changing their mind. The assistant's job is to flag consequences, then follow direction.

Part B — Designing the system for honesty and reproducibility

B1. Baseline-first, enforced rather than asserted

The forecasting work exists to answer "can a real model beat a plain one?" — and the answer was *no*, out-of-sample (Document 02). Because the rule was enforced with a real 12-fold walk-forward, the honest result survived: the simple seasonal-naive baseline is the shipped forecast, and the dashboard shows the evidence. A project that only *asserts* baseline-first would have quietly adopted the fancier model.

B2. Write assumptions into the output, not just the docs

Every simplifying assumption travels *with the data*. The route file carries its own `depot_assumption` and `max_stops_note`; the typology block carries its low-volume threshold and how many stations it excluded; the school layer carries its vintage label. A reviewer reading the JSON sees the caveat without reading this document — which is exactly where a caveat needs to be to survive.

B3. Verify against facts from outside the system

Ground-truth spot checks assert the pipeline's output against things a reviewer already knows — the Financial District is an office district, Stuyvesant Town is residential, Thanksgiving is a holiday, NYC winters have less ridership than summers — and deliberately never use the pipeline's own derived labels (that would be circular). Every real bug the project found (contaminated station coordinates, a skewed weather-zone scheme, a stale caveat) was caught by checking a claim against reality, so that habit was made a standing, repeatable check instead of an accident of investigation.

B4. Small, pure, auditable functions beat clever ones

The color logic, the cluster coloring, the elasticity math — each is a small pure function that takes plain numbers in and returns a value, testable without stubbing a map library or a browser. The clustering is k-means on interpretable unit vectors whose centers are themselves inspectable 24-hour curves. "No black boxes" was a rule, and it is what let the whole analysis be reproduced and spot-checked.

B5. Make reproduction a single real command

One entry point re-derives every artifact by running the actual pipeline steps in order and stopping on the first real failure — it does not re-import internals or wrap them in something that could claim success without running them. And the one genuine non-reproducibility (the live GBFS feed has no historical archive) is stated plainly in the reproduction script itself, not implied away.

Part C — Product decisions that kept it credible

C1. Frame the metric honestly, then complement its blind spot

Net flow is a *demand-pressure proxy*, not an observed-failure count — it cannot see the trips that did not happen. Rather than overclaim, the tool labels it as a proxy wherever it appears and pairs it with a live GBFS layer that shows the real current state. Naming a metric's limitation and then covering the gap is more credible than a single number that pretends to be complete.

C2. Report negative results as first-class findings

The model losing to the baseline, and the arrivals/departures split failing to help, are both surfaced — in the dashboard and in the docs — not buried. For a portfolio aimed at analysts, a visible negative result is stronger evidence of rigor than an unbroken string of wins.

C3. Draw the scope boundary and hold it

Per-station equity context is everywhere it is useful (detail panel, tooltips, ranked lists), but there is deliberately no map-level equity filter that recolors the whole map by equity status — a standing decision held even though the reference implementation and a later spec both assumed one. The right response to a spec that conflicts with a standing boundary is to stop and ask how to adapt it, not to silently build the thing the boundary forbids or silently skip the feature.

C4. Design the interface to be honest in place

The weather panel states next to its own sliders that it is an elasticity estimate, not a weather model; the fleet panel explains the no-diminishing-returns result where the slider is; the model-performance table shows the baseline winning. Honesty embedded at the point of interaction is harder to miss than an honesty section in a report no one opens.

The short version

> **IF YOU TAKE FOUR THINGS FROM THIS DOCUMENT**
>
> **1.** Give the assistant standing rules it enforces for you, and a durable written log so settled questions stay settled. **2.** Verify everything against reality — a real browser, a rendered screenshot, a specific measured number, a fact from outside the system — before calling it done. **3.** Build for honesty: enforce baseline-first, write assumptions into the output, report negative results, and reproduce from one real command. **4.** Re-check inherited reasoning, and let the human's current intent override past rationale — including the assistant's own.